Supplementary Material for "AutoResearch-Ledger: Transactional Revision Integrity for Executable Research Agents"

Rajesh Kumar, Waqar Ali*, Nikesh Lagun, Shaoning Zeng

This supplement contains material that supports, but is not needed to follow, the main paper: full state-machine relations, implementation details, complete benchmark tables, the 13-case historical tracked-proxy bridge study, transaction fault injection, protocols for the confirmatory study that remains unexecuted, a comparison with related systems, and reproducibility information. References to "the main text" are to the accompanying manuscript.

S1. Formal Details

A. Per-object state machines

Table S1 gives the full transition relations. Legacy labels from the earlier prototype are canonicalized on load: `verified`, `supported` and `unverified` on evidence become `provenance_verified`, `accepted` and `proposed`; `unverified` on claims becomes `unsupported`. Earlier snapshots and scripts therefore continue to load.

B. Derivation of claim and assertion states

For a claim c, the ledger builds $\mathcal{S}(c)$ from three sources: every evidence id in `Claim.evidence` that is in neither `required` nor any `support_groups` entry is a singleton set; each `support_groups` entry is one conjunctive set; `required` gives $R(c)$ and `depends_on` gives $D(c)$. Missing ids are dropped from singleton sets (a behaviour inherited from the earlier prototype, made harmless because I_1 blocks such states from committing) but retained in conjunctive sets, R and D, where dropping a missing premise would silently weaken the claim. Effective liveness of evidence is memoized per reconciliation and follows `derived_from` and run status; a lineage cycle is broken conservatively as live and reported separately by I_3. An assertion in `draft` or `outdated` keeps its state; otherwise it is `admissible` iff it binds at least one claim and every bound claim is supported or verified with registered support, and `blocked` otherwise.

Evaluating one claim costs $O(|\mathcal{S}(c)| \cdot \max|S| + |R| + |D|)$ plus lineage depth. Propagation from a rejected record visits its lineage closure, the claims and assertions linked to closure members, and (by scan) the decisions citing them; this scan is the source of the $O(D)$ propagation cost visible at 10^5 triples.

*Corresponding author: Waqar Ali (ORCID: 0000-0003-0846-7281; e-mail: waqar.uestc@yahoo.com). Rajesh Kumar ORCID: 0000-0003-0813-7485; Nikesh Lagun ORCID: 0009-0005-6372-4852.

C. Operational construct boundary

The state derivation above is deterministic only *after* the relation structure has been supplied. The implementation does not infer whether an evidence record is scientifically sufficient, whether a premise is genuinely necessary, whether two apparent alternatives are statistically independent, or whether a manuscript span is correctly bound to a claim. We therefore distinguish *representation semantics* from *construct acquisition*. The former is tested by SUP, REG, SM-CONF and TMS-XC; the latter requires independent annotation and is the primary unresolved target of Section S7. In particular, "live," "supported" and "admissible" are ledger-relative workflow labels, not truth or credibility judgements.

D. Verification tiers

`verify_evidence` evaluates: *artifact* (every declared artifact exists and matches its digest; `unchecked` if none declared), *execution* (`pass` iff the bound run is successful with a recorded environment digest; `fail` for a failed or invalidated run; `unchecked` while created), *result* (a caller-supplied checker; `unchecked` otherwise). A ledger may set a minimum number of passing tiers (`min_verification_tiers`); the default is zero so that synthetic mechanism fixtures without artifacts can remain structurally admissible. This fixture policy should not be interpreted as scientific verification: the main-text `provenance_verified` construct is explicitly narrower than scientific validity, and real tracked-proxy evidence is backed by immutable runs/artifact hashes rather than by the zero-tier fixture setting.

S2. Implementation and Integration

TRL is a Python package (`research/ledger`) of typed dataclasses (evidence, claim, decision, assertion, run), `states.py` (vocabularies and transition relations), `support.py` (liveness and claim derivation), `verification.py` (tiers), `invariants.py`, `transaction.py`, a content-addressed store with an advisory commit lock and hash-chained JSONL log, a history audit, a run-directory bridge, an adapter for ResearchLedger workspaces, and explicit pipeline hooks. Two store options were added for the scaling study: a single-pass canonical serialization shared by hash and snapshot, and re-parsing the head instead of deep-copying; both leave content hashes identical (tested). Table S2 lists integration points with the AutoResearch pipeline.

TABLE S1
PER-OBJECT STATE MACHINES. SELF-TRANSITIONS ARE ALWAYS LEGAL. CLAIM AND ASSERTION STATES ARE *derived* FROM THE SUPPORT STRUCTURE AND THE STATES OF THE OBJECTS THEY DEPEND ON (THE MAIN TEXT); EVIDENCE, DECISION AND RUN STATES CHANGE BY EXPLICIT OPERATION.

Object	States	Legal transitions and notes
Evidence	proposed, accepted, provenance_verified, rejected, superseded	proposed → accepted, provenance_verified, rejected, superseded; accepted → provenance_verified, rejected, superseded; provenance_verified → accepted (demotion after a failed re-verification), rejected, superseded. `rejected` and `superseded` are terminal: reviving an invalidated result requires a *new* record, not a state change (e.g. superseded → provenance_verified is illegal).
Claim	unsupported, supported, verified, contested, stale	Derived. Every transition between distinct states is reachable (the main text, SM-CONF); the operative constraint is that the stored state equal the derivation (I_2). `stale` = a registered support structure lost a necessary member; `unsupported` = no registered live sufficient set; `contested` = live support and live contradiction.
Decision	proposed, decided, revisiting, revised, abandoned	proposed → decided, abandoned; decided → revisiting, abandoned; revisiting → revised, abandoned; revised → revisiting, abandoned. A decided or revised decision may not cite evidence that is no longer live (I_4).
Assertion	draft, admissible, blocked, outdated	draft → admissible, blocked, outdated; admissible ↔ blocked; admissible, blocked → outdated (terminal). Admissible/blocked are derived from the bound claims.
Run	created, successful, failed, invalidated	created → successful, failed; successful → invalidated; failed and invalidated are terminal.

TABLE S2
AUTORESEARCH-LEDGER INTEGRATION POINTS. TRL IS A STATE-CONTROL LAYER, NOT AN ADDITIONAL LANGUAGE-MODEL ROLE.

Pipeline region	Ledger object	Transactional role
Literature discovery	Evidence	Bind source identity/locator; support or contradict explicit claims.
Experiment execution/analysis	Evidence + artifact hash	Bind numerical findings to a successful run artifact; later replacement supersedes rather than overwrites.
Knowledge synthesis	Claim	Maintain bidirectional support/contradiction relations and deterministic claim status.
Research decision	Decision	Record evidence dependencies; invalid evidence moves the decision to `revisiting`.
Paper writing/revision	Manuscript reference	Map paper assertions to claim IDs so stale claims cannot silently survive a research-state update.
Final export	Global invariant gate	Commit only when provenance, supersession, artifact, and manuscript-admissibility invariants hold.

a) Bridge status vocabulary: The core TRL specification and the ResearchLedger v2.2 bridge are separate layers with different native status names. Core TRL derives claim labels `unsupported`, `supported`, `verified`, `contested` and `stale`. ResearchLedger v2.2 derives `hypothesis`, `provisional`, `supported`, `mixed`, `contradicted` and `withdrawn`. The historical tracked-proxy study reports the latter labels because it executes the ResearchLedger bridge directly. In that bridge, `hypothesis` is returned when no currently sufficient live support path has positive support level; `provisional` corresponds to a live path with observed/checked evidence; and `supported` corresponds to a live path whose members are verified. The reported `supported→hypothesis→supported` lifecycle therefore demonstrates loss and restoration of bridge support, not a transition in the core TRL state table. Keeping the vocabularies explicit avoids treating two independently evolved implementation labels as identical constructs.

b) Test inventory: The integrated ResearchLedger v2.2 validation build passed 206/206 automated tests across 22 modules in five non-overlapping batches: 88 tests for models/validator/revision/revision CLI/tracked trajectories/paper validation/study tooling; 68 for atomicity/CLI/errors/environment/evidence/examples/hooks/indexing/migration/reporting; 14 reproduction tests; 20 runner tests; and 16 skill-wiring/trace/workspace tests. This supersedes the earlier separately reported 56-test TRL and 93-test ResearchLedger counts for the code exercised by the updated historical-validation study. The run used the same source tree as the tracked-trajectory results. `ruff` and `mypy` were not installed in that execution environment, so no new lint or static-type result is claimed.

S3. CONFORMANCE BENCHMARKS

All six benchmarks are deterministic, network-free and generated from fixed seeds. They test conformance of the implementation to its specification.

a) RSM-1000: Each trial starts from a valid state of 20 manuscript-referenced claims, 20 evidence records and 20 evidence-dependent decisions; one mutation (seed 7) is drawn from evidence rejection, supersession, a broken reciprocal link, a dangling deletion, or a decision left active after its evidence is rejected. Rejection, supersession and the stale-decision case are legitimate; the other two are structurally corrupt. Four mechanisms are compared: raw mutable state, post-hoc claim repair, transactions without propagation, and full TRL. Table S3 shows that raw state left 2.005 violations per trial and let stale claims stay active in 79.9% of evidence-invalidating trials; post-hoc repair removed stale claim labels but still left 1.206 violations; transactions without propagation were safe but rejected every legitimate update; full TRL achieved zero violations, zero escape and full acceptance and rejection.

b) RSM-SEQ-2500: 100 episodes of 50 triples and 25 unique mutations (seed 17). Raw mutation and post-hoc repair ended no episode consistent (50.19 and 30.02 residual violations); full TRL ended all 100 consistent (Table S4).

TABLE S3
RSM-1000 MECHANISM RESULTS (1,000 DETERMINISTIC MUTATION TRIALS; SEED 7). MCR: MUTATION CONTAINMENT RATE; SCER: STALE-CLAIM ESCAPE RATE; VA: LEGITIMATE-UPDATE ACCEPTANCE; IR: INVALID-UPDATE REJECTION. THESE RESULTS TEST STATE SEMANTICS, NOT GENERAL SCIENTIFIC COMPETENCE.

Mechanism	MCR ↑	Residual invariant violations/trial ↓	SCER ↓	VA ↑	IR ↑
Raw mutable state	0.0%	2.005	79.9%	100%	0%
Post-hoc claim repair	0.0%	1.206	0.0%	100%	0%
Transaction, no propagation	100%	0.000	0.0%	0%	100%
Full TRL	**100%**	**0.000**	**0.0%**	**100%**	**100%**

TABLE S4
RSM-SEQ-2500 CUMULATIVE MUTATION RESULTS (100 EPISODES, 25 MUTATIONS EACH; 2,500 TOTAL; SEED 17).

Mechanism	Clean episodes ↑	Final violations ↓	Final SCER ↓	VA ↑	IR ↑
Raw mutable state	0%	50.19	100%	100%	0%
Post-hoc claim repair	0%	30.02	0%	100%	0%
Transaction, no propagation	100%	0.00	0%	0%	100%
Full TRL	**100%**	**0.00**	**0%**	**100%**	**100%**

TABLE S5
RSR-500 STALE-WRITER RACE RESULTS.

Mechanism	Conflict detected	Lost update	Both preserved
Last-write-wins	0%	100%	0%
OCC, no retry	100%	0%	0%
OCC + rebase	**100%**	**0%**	**100%**

TABLE S6
RLV2-400 STRICT-VALIDATOR CHALLENGE (50 TRIALS PER CONDITION; SEED 20260916).

Condition	Strict detection	Target rule hit
Clean control	0/50	–
Broken graph edge (RL101)	50/50	50/50
Artifact hash drift (RL202)	50/50	50/50
Failed run cited (RL211)	50/50	50/50
Experimental evidence without run (RL111)	50/50	50/50
Superseded dependency (RL302)	50/50	50/50
Claim-status drift (RL310)	50/50	50/50
Broken manuscript provenance (RL401)	50/50	50/50

c) RSR-500: Two valid claim additions begin from the same predecessor (500 trials, seed 31). Last-write-wins silently lost one update every time; optimistic concurrency detected every stale writer; with one rebase-and-retry both updates survived in all 500 trials (Table S5). This is a single-host test, not distributed consensus.

d) HT-250: Ledgers with eight transactions receive one of five partial-history corruptions (transaction edit, snapshot edit, snapshot deletion, head rollback, log truncation), 50 per class (seed 23). All 250 were detected; 25 untouched controls produced no false positive. The chain is unsigned, so a complete-history rewrite is out of scope.

e) RLV2-400: 400 network-free ResearchLedger v2 workspaces (50 per condition): 50 clean and 350 with one injected failure from seven documented rule classes. Strict validation accepted all controls and rejected all 350 corruptions with the expected rule code (Table S6); the challenge is derived from the same rules it tests.

TABLE S7
DRS-1400 DEPENDENCY-REGISTRATION SENSITIVITY (200 TRIALS PER p_{MISSING}, 20 CLAIMS/TRIAL; SEED 41). LEDGER SCER IS THE METRIC DEFINED IN THE MAIN TEXT, COMPUTED ONLY OVER REGISTERED EDGES; GROUND-TRUTH SCER ADDITIONALLY COUNTS CLAIMS WHOSE TRUE EVIDENCE WAS INVALIDATED BUT NEVER LINKED.

p_{missing}	Residual violations	Ledger SCER ↓	Ground-truth SCER	Missing-group escape
0.0	0.000	0.0%	0.0%	n/a
0.1	0.000	0.0%	9.9%	100%
0.2	0.000	0.0%	20.0%	100%
0.4	0.000	0.0%	40.7%	100%
0.6	0.000	0.0%	59.1%	100%
0.8	0.000	0.0%	80.6%	100%
1.0	0.000	0.0%	100.0%	100%

f) DRS-1400: For 20 claims per trial, the edge from a claim's true evidence is left unregistered with probability $p_{\text{missing}} \in \{0, \ldots, 1\}$ and a decoy support keeps the claim well-formed; all true evidence is then rejected in one legitimate transaction (200 trials per point, seed 41). The ledger raises no violation and its own escape metric stays 0%, while ground-truth escape equals p_{missing} (Table S7); this is a controlled measurement of a stated boundary condition, not an estimate of real registration quality. The REG sweep of Section S4 refines it.

S4. SUPPORT-SET AND REGISTRATION DIAGNOSTICS

a) Generator: Each synthetic trajectory has 8 claims, about 14 evidence records, 1–3 runs, 3–5 decisions and one assertion per claim (a fifth of assertions bind a second claim). Claim templates: singleton support (30%), conjunctive group of two or three records (25%), two alternative groups (20%), singleton plus required premise (15%), singleton plus non-epistemic dependency (10%). A third of evidence is derived from earlier evidence and about half of the rest from a run. Three evidence records with downstream edges are rejected per trajectory; 815 events across 300 trajectories have non-empty impact (seed 61). REG draws 100 trajectories per point (seed 62) and corrupts the registered graph by dropping each gold edge with probability p, flattening groups and premises to independent `supports`, or adding spurious `supports`

TABLE S8
SUP-300 BY CLAIM TEMPLATE: RECALL OVER INVALIDATED CLAIMS OF THAT TEMPLATE (COUNT IN SECOND COLUMN) AND FALSE-INVALIDATION RATE OVER STILL-JUSTIFIED CLAIMS OF THAT TEMPLATE.

Template	Affected	Binary	EG-R	Full	EG-R FIR	Full FIR
Singleton support	228	0.728	1.000	1.000	0.000	0.000
Conjunctive group	417	0.000	1.000	1.000	0.000	0.000
Alternative groups	88	0.000	1.000	1.000	0.303	0.000
Required premise	215	0.000	1.000	1.000	0.000	0.000
Non-epistemic dependency	162	0.383	1.000	1.000	0.000	0.000

edges. The gold structure is authored by the generator, so these are mechanism diagnostics.

b) Identified versus updated: The provenance graph without transactions identifies every affected claim, decision and assertion (recall 1.000, FIR 0.070) but updates none; measured over stored state its recall is 0.000 and its stale-assertion escape 1.000. Both views are computed for every condition in the released results.

S5. State-Machine Conformance Details

The first fuzzing pass (60 episodes) reported 128 "legitimate" operations rejected as illegal transitions. Most were fuzzer errors—accepting, superseding or rejecting evidence that was already dead—which the transition check correctly refused. Two were genuine inconsistencies in our specification: re-supporting a `stale` claim with a not-yet-accepted record derived `stale→unsupported`, and declaring an unaccepted required premise on a supported claim derived `supported→unsupported`. Because claim states are derived, we made every claim transition legal and kept the derivation check (I_2) as the operative constraint, then added directed tests. The final run (150 episodes $\times$ 40 steps, seed 53) attempted 4611 operations and committed 4582; no legitimate operation was rejected as an illegal transition, and all 29 rejections were `stale_decision`. Unexercised table edges: claim: contested→verified, claim: stale→verified, claim: verified→supported, claim: verified→unsupported. Illegal-write fuzzing: 500 of 500 forbidden evidence, decision, assertion and run writes rejected; 100 of 100 claim-state writes disagreeing with the derivation rejected as drift.

S6. Scaling Study

Scale is the number of claim/evidence/decision triples (three records each). Density d is the number of evidence links per claim; for $d \geq 4$ half of them form a conjunctive group, one is a required premise, one a non-epistemic dependency, and every tenth evidence record is derived from its predecessor. Components are timed separately on a warm interpreter: propagation, invariant checking, serialization, hashing, snapshot write, cold load, one full incremental revision (begin, reject, commit) and history audit. Two store configurations are compared on identical states: the earlier path (indented snapshot written after a separate hashing pass; deep-copy in `begin`) and a path that serializes once in canonical compact form and re-parses the head twice. A third configuration disables `fsync` (keeping atomic rename) to separate storage-flush cost from CPU cost; it is used only in this study. Absolute times are machine-specific: the earlier prototype's code, rerun on the same laptop, takes 183 ms per commit at 1,000 triples (against 51.8 ms on the hardware that produced the earlier published numbers), and medians at 10^3 triples varied by up to 3× with background load, so only $\geq 10^4$ triples are interpreted. Auditing a 25-transaction history on a 1,000-triple ledger takes about 3.1 s and 18–29 MB, because every transaction stores a full snapshot. Table S10 lists all measured points.

S7. Protocols for Studies Not Executed

A. RRT-100: real research revision trajectories

a) Selection: 50–100 complete AutoResearch runs, each reaching a manuscript draft with at least five claims and two manuscript assertions; workspaces are frozen before annotation; stratify by domain and agent configuration; do not filter on how good TRL's graph looks. Conversion `ingest-ledger` produces the *registered* graph, which annotators never see.

b) Blinding and annotators: At least two annotators per trajectory and at least three on 25 trajectories, technically competent in the domain and not authors of TRL or the evaluated agent. They receive object texts, artifact links and a candidate-pair worksheet (`blind`), with no registered graph, status labels, mutation targets or impact labels.

c) Relations and counterfactual decision rule: Annotators first ask: *if source object E were shown invalid while all other registered information stayed fixed, what would have to happen to target C?* They then apply Table S11. The procedure intentionally separates evidentiary sufficiency from revision dependence and guards against marking every co-cited or merely associated record as support.

d) Proposed taxonomy for D, R and $\mathcal{S}$ (untested): The codebook is a starting point, not a validated ontology; annotator agreement is precisely what the pilot must measure, and $\kappa < 0.6$ on any category means that category is not sufficiently well defined for confirmatory scoring without the preregistered refinement. Record an observation (run result, citation or derivation) in $\mathcal{S}(c)$ if it would by itself, or with named partners, justify the claim. Put an observation in $R(c)$ if every sufficient route requires it (for example the dataset on which all routes were computed). Put an item in $D(c)$ if its change requires re-examination but it is not an observation bearing on the proposition: code/library versions, preprocessing/data transformations, analysis scripts, statistical model specifications, model checkpoints, instrument calibration or auxiliary theoretical assumptions. A replication is an alternative set if it would suffice alone, a member of the same group if the claim needs both, and otherwise not registered as support. Statistically dependent evidence is not representable as independent alternatives; register shared lineage or a common R item and flag the case as a limitation. Partial support and evidentiary strength are not represented. TRL also has no semantic claim-edit operation or general relation deletion: a materially reworded claim becomes a new record with relations re-established. Propagation after claim edits and discovery of latent dependencies are not implemented.

TABLE S9

REG: REGISTRATION CORRUPTION ON SYNTHETIC TRAJECTORIES (100 PER POINT; SEED 62). P, R: TYPED EDGE PRECISION AND RECALL AGAINST GOLD; SSC: SUPPORT-SET COVERAGE; RECALL: INVALIDATED CLAIMS RECOVERED BY FULL TRL; DEC.: DECISION-REVISION RECALL; ESCAPE: INVALID ASSERTIONS LEFT ADMISSIBLE (FULL TRL AND TRL WITH BINARY EDGES).

Corruption	P	R	SSC	Claim recall	Dec. recall	Escape (full)	Escape (binary)
None	1.000	1.000	1.000	1.000	1.000	0.000	0.774
Drop 5%	1.000	0.947	0.890	0.903	0.947	0.172	0.767
Drop 10%	1.000	0.902	0.818	0.845	0.881	0.242	0.747
Drop 20%	1.000	0.811	0.654	0.715	0.807	0.422	0.827
Drop 30%	1.000	0.704	0.539	0.623	0.710	0.547	0.816
Drop 40%	1.000	0.595	0.394	0.555	0.512	0.668	0.858
Drop 50%	1.000	0.501	0.307	0.395	0.464	0.809	0.890
Flatten 50% of conjunctions	0.859	0.864	0.537	0.541	1.000	0.454	0.810
Flatten 100% of conjunctions	0.733	0.733	0.448	0.404	1.000	0.571	0.757
Spurious 10%	0.921	1.000	0.615	0.746	1.000	0.234	0.862
Spurious 30%	0.790	1.000	0.209	0.380	1.000	0.612	0.944

TABLE S10

SCALING STUDY, ALL MEASURED POINTS (MEDIAN OF 3–5 REVISIONS). "DENSITY" IS EVIDENCE LINKS PER CLAIM. PROP.: DEPENDENCY PROPAGATION (MS); OTHER TIMES IN SECONDS. "REVISION" IS BEGIN + REJECT + COMMIT ON A COMMITTED ARCHIVE. THE EARLIER STORE WAS NOT RUN AT 10^5 TRIPLES.

Triples	Density	Store	Prop. (ms)	Invariants	Commit	Revision	Cold load	Snapshot (MB)
10^4	1	Earlier	7.5	0.15	2.46	3.69	0.51	11.3
10^4	1	Single-pass	5.1	0.12	0.89	1.99	0.47	7.1
10^4	1	Single-pass, no fsync	22.5	0.55	3.43	6.90	1.35	7.1
5×10^4	1	Earlier	31.2	0.71	11.42	18.04	2.04	56.6
5×10^4	1	Single-pass	29.3	0.76	5.52	8.75	1.96	35.8
5×10^4	1	Single-pass, no fsync	24.9	0.61	5.12	12.72	7.21	35.8
10^5	1	Single-pass	195.2	5.33	42.86	65.47	11.57	71.6
10^5	1	Single-pass, no fsync	222.1	5.22	42.30	65.78	14.24	71.6
10^4	5	Earlier	21.5	0.52	6.13	10.15	1.26	14.2
10^4	5	Single-pass	5.3	0.15	1.45	1.79	0.47	8.3
10^4	5	Single-pass, no fsync	4.4	0.14	1.35	1.85	0.44	8.3
5×10^4	5	Earlier	69.8	2.31	32.26	50.92	6.04	71.1
5×10^4	5	Single-pass	56.2	2.36	15.99	25.19	5.84	41.8
5×10^4	5	Single-pass, no fsync	67.5	2.15	15.88	25.23	5.79	41.8
10^4	20	Earlier	6.4	0.37	3.30	4.97	0.55	22.3
10^4	20	Single-pass	4.9	0.35	2.05	2.54	0.57	12.5
10^4	20	Single-pass, no fsync	4.8	0.35	2.03	2.54	0.56	12.5

TABLE S11

PREREGISTERED RELATION-ANNOTATION CODEBOOK (UNTESTED). THE DECISION RULES ARE OPERATIONAL HYPOTHESES WHOSE INTER-RATER RELIABILITY MUST BE MEASURED BEFORE THE RESULTING GRAPH IS TREATED AS A STABLE REFERENCE CONSTRUCT.

Relation	Positive counterfactual criterion	Do not use when
`supports`	E alone would be sufficient for C under the stated claim scope	E merely co-occurs with the claim, improves confidence, or is required only with another named record
`support_group`	E is one member of a named set whose members are jointly sufficient and individually insufficient	the records are independent alternative replications; use separate alternative groups instead
`required_for`	C cannot remain justified without E regardless of which sufficient route is used	E is only one member of one sufficient route, or its change merely warrants re-examination without itself bearing on the proposition
`depends_on`	invalidating E requires re-examining the target, but E is not itself evidentiary support	E is an observation bearing directly on the claim; then use a support relation or required premise
`contradicts`	live E supplies evidence against the claim	E is merely absent support or a failed computation with no negative evidentiary content
`derived_from`	the target artifact/evidence was computed from, transformed from or produced by the source	the relation is only conceptual association or co-citation
claim→assertion	the manuscript span expresses or relies on the claim such that loss of the claim requires revision of the span	the claim is merely topically related but the sentence remains valid without it

e) Mutations and impact labels: Three to five per trajectory: retraction, supersession, lineage-root invalidation and at least one control with empty true impact, sampled by stratified out-degree in the *gold* graph and fixed before any system runs. Annotators independently label which claims, decisions and assertions are no longer justified *without consulting the graph*; the exact-match rate between these labels and the oracle's graph-derived impact is reported, since disagreement means the vocabulary omits something annotators judge to matter.

f) Adjudication and agreement: A strict-majority draft and the disputed edges are produced by `adjudicate`; a third-party adjudicator writes the gold graph. Agreement is Cohen's or Fleiss' κ over the pair universe [1], [2], typed edge F_1 and support-structure Jaccard. If $\kappa < 0.6$ on edge presence, we report that "true dependency" is not sufficiently well defined under this vocabulary; one pre-registered refinement round is allowed before adjudication and must be reported.

g) Analysis plan: The unit of analysis is the trajectory; ratio-of-sums cluster bootstrap with 95% percentile intervals; the primary comparison is full TRL against each other condition on stale-assertion escape and false-invalidation rate, paired by trajectory (sign-flip permutation, Holm correction); registration→protection is a regression of per-trajectory escape on $1 - \mathrm{R}_{\mathrm{dep}}$ with the fraction of escapes attributed to unregistered, flattened and mistyped edges; every metric is recomputed against each annotator's graph as an annotation-uncertainty sensitivity.

h) Conditions: Matched-registration is primary: all graph-based conditions receive the same registered graph. A native-registration condition is admissible only for systems that can genuinely be executed. EG-R is a reconstruction of the mechanism reported for EviGraph and every table row carries that label.

B. Human auditability study

Within-subject, 15–30 participants with research-engineering experience, recruited independently of the authors. Condition A: the agent's output directory as produced today. Condition B: the same plus TRL's claim→evidence→run→artifact trace, current states and export-gate reasons, without indicating the faulty item. Each participant audits four tasks (two per condition), each a frozen trajectory in which one evidence record was invalidated after the manuscript was drafted (tampered artifact, superseded run, retracted citation, lineage-root invalidation), assigned by Latin square to balance condition, trajectory, fault type and position; 15 minutes per task after 10 minutes of training. Outcomes: time to locate the faulty evidence, affected-claim precision and recall, stale manuscript statements missed (primary), and confidence (secondary). Primary analysis: paired sign-flip permutation test (exact for $n \leq 20$), percentile-bootstrap interval of the mean paired difference (geometric-mean ratio for time), paired d_z and Holm correction over three primary outcomes; exact McNemar for the located rate. Threats: the interface is newly learned in B, participants know a fault exists in every task, and tasks are synthetic edits of real trajectories.

S8. Comparison with Related Systems

Table S12 is a descriptive comparison of what each cited paper reports as its core mechanism, compiled from its published description. "NR" means not reported as a core mechanism; it does not prove an implementation could not support it. None of these systems was run head-to-head with TRL.

A. Correspondence with truth-maintenance and provenance models

Table S13 states how each TRL construct maps onto justification-based (JTMS) and assumption-based (ATMS) truth maintenance and onto positive-Boolean provenance, and where the mapping ends. The mapping is exact for the monotone, single-context fragment; the rows marked "none" indicate either omissions from TRL (no non-monotonic justifications, no nogoods, no context lattice) or systems features added by TRL (atomic commit, tamper-evident history, typed lifecycles, export gate) that are not part of the classical JTMS/ATMS formalisms compared here. This is a scope statement about the cited formalisms, not a universal claim about every truth-maintenance implementation.

a) Executable check (TMS-XC): `eval/tms_crosscheck.py` implements the JTMS as a forward-fixpoint labelling and the ATMS as environment labels, each written without reference to the oracle or to TRL, and compares both, the oracle and full TRL on the same graph and mutation. Table S14 gives the results; the graph is the gold graph or a registered graph corrupted as in REG. The classification of each TRL divergence is computed by the script, not assigned by hand.

The two divergence causes are properties of the implementation, not of the semantics. *Already blocked*: TRL reports state transitions, so an assertion that cites two claims, one of which never had support, is not reported when the other loses support; it was not admissible before the revision either. *Pair collision*: in `support.py` an evidence record named in a support group or required set is excluded from the singleton alternatives of that claim, so a second, mistyped `supports` edge on the same pair replaces rather than adds an alternative, and TRL invalidates a claim that the oracle keeps. We have not changed this behaviour, because the frozen results in the paper were produced with it; a registration layer could reject a second relation on an existing pair.

S9. Historical Revision Evidence and Ethical Considerations

A. Executed historical-semantic replay, tracked proxy trajectories and transaction fault injection

This subsection reports development evidence collected after the main mechanism experiments, using genuine historical revisions rather than only generator-authored fixtures. It is developer-run and non-blind: semantic fixtures, dependency bindings and the paper-agent proxy were constructed by the developers rather than independent annotators, so it strengthens implementation and integration confidence but does not satisfy the preregistered confirmatory study. It is summarized in the main paper's Results and referenced there as Supplement S9.

a) Historical semantic replay: Table S15 lists 13 revisions across nine repositories, verified against public repository metadata on 2026-09-23 (revision identity and changed-file counts only; the authored dependency graph and downstream claim impact are not verified by this audit). For each case we replayed the smallest changed semantic unit on a fixture derived from the commit. All 13 reproduced the documented old/new difference. RH01, RH03, RH07 and RH10 are direct code or configuration replays reported qualitatively as "differs as documented"; the remaining cases carry a measured before/after value. For RH04, an earlier description of this fixture stated that the corrected Geary's-C analytic variance crossed the 0.05 significance threshold; the measured values are approximately 0.0444 before correction and 0.0209 after, both below 0.05, so this is a material p-value shift and *not* a threshold crossing,

TABLE S12
NOVELTY BOUNDARY RELATIVE TO CLOSELY RELATED 2026 ASSURANCE ARCHITECTURES. "NR" MEANS NOT REPORTED AS A CORE MECHANISM IN THE CITED WORK; "PARTIAL" DENOTES RELATED BUT DIFFERENT SEMANTICS.

System	Operational evidence state	Response to evidence change	Global transaction semantics	Decision provenance	Auditable revision mechanism
Paper2Agent [3]	Tested MCP tools + paper resources	Execute–diagnose–repair for tool/repository failures; no persisted claim-revision semantics reported	NR	NR	Tool tests + code references
ScientistOne [4]	Chain-of-Evidence	NR	NR	NR	Post-hoc CoE audit
EviGraph [5]	Typed evidence graph	Downstream subgraph regeneration	Partial: graph checkpoint/repair	NR	Graph checkpointing
ARIS [6]	Claim ledger + raw evidence	Partial: verdict propagation	NR	NR	Evidence-to-claim audit cascade
Continuity Kernel [7]	Generic versioned agent state	Exact-predecessor activation	Atomic commit/reject/quarantine/defer	Generic authority/lineage	Activation receipts + lineage
AutoResearch-Ledger	**Typed claim/evidence/decision state**	**Deterministic stale-dependency propagation**	**Validated commit/rollback**	**Explicit evidence-linked decisions**	**Hash-chained history + RSM benchmark**

TABLE S13
MAPPING OF TRL CONSTRUCTS TO TRUTH MAINTENANCE AND PROVENANCE. "LIVE" IS THE DERIVED PREDICATE THAT DECIDES CLAIM STATE.

TRL	JTMS [8]	ATMS [9]	Provenance [10], [11]
Base evidence, run	premise or enabled assumption node	assumption	annotation variable of a source tuple
Sufficient set	in-list of one justification	one environment	one witness (monomial)
$S(c)$ (family of sufficient sets)	the justifications of the claim node	the claim's label (minimal environments)	minimal witness basis; positive-Boolean polynomial
Required premise, non-epistemic dependency	in-list member of every justification	element of every environment	factor common to every monomial
Lineage (`derived_from`)	justification of the derived node	environment includes the parents' environments	provenance of a derived tuple
Reject or supersede evidence	retract the assumption, relabel	remove it from the context	deletion propagation: valuation of the variable to false
Claim live	node is IN	some environment $\subseteq$ context	polynomial evaluates to true
Decision, assertion	node whose in-list is its dependencies	label of the union of its dependencies' environments	derived output
Contradicting evidence	contradiction node (none: TRL only marks `contested`)	nogood (none)	none (negation is outside the semiring)
Non-monotonic (OUT) justification	supported	not applicable	none
Several contexts	one current context	lattice of contexts	one valuation at a time
Atomic commit, rollback, hash-chained history, typed lifecycles, export gate	none	none	none

TABLE S14
TMS-XC: REVISION EVENTS ON WHICH EACH FORMULATION'S AFFECTED CLAIMS, DECISIONS AND ASSERTIONS EQUAL THE INDEPENDENT ORACLE'S (300 SYNTHETIC TRAJECTORIES, SEED 61). BLOCKED: ASSERTION ALREADY BLOCKED BEFORE THE REVISION; COLLISION: RELATION COLLISION ON AN (EVIDENCE, CLAIM) PAIR; BOTH CLASSIFY FULL TRL'S DIVERGENCES.

Graph	Events	JTMS	ATMS	TRL	Blocked	Collision
Gold	900	900	900	900	0	0
Drop 20%	900	900	900	875	25	0
Flatten	900	900	900	900	0	0
Spurious 30%	900	900	900	876	0	24

TABLE S15
EXECUTED HISTORICAL-SEMANTIC REPLAY: 13 REVISIONS, NINE REPOSITORIES. REPLAY MODE ABBREVIATIONS: CODE (DIRECT CODE/EXPRESSION REPLAY), CONFIG (CONFIGURATION/DEFAULT REPLAY), NUMERIC (NUMERIC-FORMULA OR COVERAGE REPLAY), LOADER (LOADER/FILTER-SEMANTICS REPLAY).

Case	Repository	Mode	Observed old $\rightarrow$ new
RH01	POP-TOOLS	config	differs as documented
RH02	scanpy	numeric	target sum 15.0 $\rightarrow$ 20.0
RH03	scanpy	config	differs as documented
RH04	squidpy	numeric	analytic p 0.0444 $\rightarrow$ 0.0209
RH05	TabPFN	numeric	class-1 prob. 0.700 $\rightarrow$ 0.206
RH06	TabPFN	numeric	feature coverage 9 $\rightarrow$ 12/12
RH07	SAELens	code	differs as documented
RH08	Compass	code	runtime error $\rightarrow$ valid PCA
RH09	scvi-tools	loader	rows 33 $\rightarrow$ 30 (surrogate)
RH10	seurat	config	differs as documented
RH11	squidpy	numeric	permutation result invariant to chunking after fix
RH12	scikit-learn	numeric	degenerate quantile vector $\rightarrow$ non-degenerate (500 nonzero)
RH13	SAELens	code	BOS id 0 dropped $\rightarrow$ preserved

and we correct that description here. RH09 executes the historical filtering predicate on a surrogate batch table rather than the original H5AD file. A supplemental PyMC underflow diagnostic was excluded from this count because the installed PyMC/ArviZ combination could not be imported in the runtime used.

b) Ledger-integrated tracked proxy trajectories: Each of the 13 cases was next executed as two immutable ResearchLedger runs through `validation_study/trajectory_runner.py`. The old run produced an artifact whose hash was captured in the run manifest, was promoted to verified bridge evidence, and supported claim `C001`. The genuine historical revision identity

TABLE S16
TRACKED PROXY LIFECYCLE OVER THE 13 HISTORICAL REVISIONS. THE EXECUTABLE IS A DETERMINISTIC PAPER-AGENT PROXY, NOT A PAPER2AGENT MCP; BINDINGS ARE DEVELOPER-AUTHORED.

Outcome	Result
Different old/new artifact hashes	13/13
Initial verified evidence + supported claim	13/13
Supersession produced `hypothesis`	13/13
Explicit old-evidence manuscript marker stale	13/13
Verified replacement restored `supported`	13/13
Replacement marker cleared stale usage	13/13
Final validation errors	0/13
Median transaction-only time to safe state	9.6 ms
Median recovery bookkeeping after new output exists	15.0 ms

was then used as the reason for transactionally superseding the old evidence; the claim lost its currently sufficient support path and moved to ResearchLedger's native `hypothesis` state. The revised semantic fixture was executed as a new immutable run, attached as replacement evidence, verified, and the claim returned to `supported`. These labels belong to the bridge vocabulary documented in Section S2 and are not silently equated with the core TRL claim labels. A manuscript marker explicitly naming the old evidence was audited before revision, after supersession and after recovery; stale usage appeared after supersession and disappeared only after the marker was changed to the replacement evidence.

Each case retains the two immutable run manifests, artifact hashes, three committed revision journals (old-evidence verification, supersession, replacement verification), audit snapshots and timing fields. Median old/new proxy executions were approximately 2.9–3.0 s and were dominated by process/environment capture. These timings are local implementation diagnostics and exclude real paper-agent generation or the scientific computation cost of an arbitrary repository.

c) Transaction fault injection: We fault-injected the multi-file revision transaction under six deterministic checks, all passing: (1) an ordinary two-entity revision commits; (2) a forced write failure on the second entity restores exact pre-image hashes and marks the transaction `rolled_back`; (3) a simulated process crash during the `applying` phase is recovered from the journal, restoring the pre-image; (4) removing one of two alternative sufficient support paths for a claim leaves it `supported`; (5) an 11-file transaction faulted at every one of its 11 entity-write positions restores all original file hashes in 11/11 trials; (6) two independent revision CLI processes launched concurrently both commit, leaving both evidence/claim pairs in the expected final states. These checks support the implemented local atomicity/recovery protocol under the tested fault model; they are not a distributed-database durability claim and do not cover storage-device failure, network partitions or hostile writers.

d) Authored structural pilot and repository-clustered bootstrap: On a separate authored 10-event pilot mapping the first ten historical revisions to controlled dependency graphs (distinct from SUP-300's 815 generator-authored events), support-set provenance and full TRL both reach exact 10/10 impact identification; flat binary edges reach 6/10; unconditional downstream reachability and unconditional full rebuild reach 4/10 exact matches, over-invalidating claims that retain an alternative sufficient support path (claim false-invalidation rate 1.0 for both, against 0.0 for support-set provenance and full TRL). A repository-clustered bootstrap (10,000 replicates, 8 repository clusters) gives a median TRL-minus-binary-edge claim-recall difference of 0.40 (95% percentile interval $[0.111, 0.700]$) and a median TRL-minus-support-set-provenance exact-match difference of 0.0 (interval $[0.0, 0.0]$), i.e. support-set provenance alone already matches full TRL on this pilot. Because the ten dependency graphs and gold impact sets were authored for mechanism development rather than independently annotated, this interval is a descriptive sensitivity analysis over the pilot, not external-validation uncertainty, and it should not be read as a confidence interval on TRL's advantage over binary edges in general. The pilot result is consistent with the main paper's SUP-300 support-set results: it attributes impact-identification gains to AND/OR support semantics rather than to the transaction layer, whose independently tested contribution is atomicity, rollback, crash recovery and auditable history (above).

e) Full test suite: The v2.2 validation bundle's complete automated test inventory passed 206/206 tests across 22 modules in non-overlapping batches because a monolithic invocation exceeds the runtime's per-command time limit; no test module was omitted. This is software-conformance evidence for the source tree exercised by the semantic replay, tracked trajectories and transaction tests, not a scientific-validity result.

f) Interpretation boundary: This evidence is not independent external validation. It strengthens confidence that genuine historical semantic changes can be carried through the implemented immutable-run, evidence, claim, transaction and manuscript-audit lifecycle, but the executable tool is a deterministic proxy, and the dependency/claim bindings, fixtures and fault schedule were authored by the developers. Two uncertainties therefore remain distinct: *construct validity* (whether independent experts can reproducibly assign the proposed dependency types and impact labels) and *end-to-end agent validity* (whether a native executable paper agent registers/preserves those dependencies under real revision). The decisive study is a full old-commit → generated executable paper agent (for example a Paper2Agent MCP) → frozen outputs/claims → historical correction → regenerated/re-executed agent, evaluated against two blind independent annotators plus adjudication. The preregistration requires at least 60 revision events from at least 15 repositories, matched- and native-registration analyses, strong logical baselines and repository-clustered uncertainty. Blind packets for the current cases remain unfilled study materials; no human labels or agreement statistic are fabricated.

B. Ethical considerations

Automated paper generation risks low-quality, misleading or plagiarized submissions. AutoResearch includes execution, citation and evidence-alignment checks before artifact promotion, and TRL adds deterministic provenance and revision-state constraints; these reduce particular failure modes but do not establish correctness, novelty or ethical acceptability.

TABLE S17
FROZEN IDENTIFIERS FOR THE REVISED TRL MECHANISM STUDY.

Artifact	Identifier
AutoResearch base	3277c5381b08dc7ec93374fa869784fb5397e3c7
ResearchLedger v2.2 validation ZIP	97e016dfb8a6d75c3c1564110af005b13aaf3ba7
Uploaded	348fe35aa216ea008bca4ae4
ResearchLedger base ZIP	c5c9f24d21484b343b5e0b265202a0583c19fc90
TRL code v13 ZIP SHA-256	1843e806f682447d06521c30
RSM output	7c66c55fda0f333bca4027eb42f43c575923f8af
Scaling output	0a70d2fdfa3411be1b16eba6
Sequential output	code_patch/benchmark_results.json
Race output	code_patch/scaling_results.json
Tamper output	code_patch/sequence_results.json
DRS-1400 output	code_patch/concurrency_results.json
RLV2-400 output	code_patch/tamper_results.json
Integrated v2.2 tests	eval/dependency_registration_results.json
Bridge smoke test	eval/researchledger_v2_validator_results.json
Unit tests	206/206 passed across 22 modules
	integration/test_bridge.py
	code_patch/tests/test_research_ledger.py

Users could still misuse the system to mass-produce superficial papers; we recommend human oversight and treating generated manuscripts as drafts. A ledger makes stale dependencies visible and preserves a reviewable history of evidence changes, at the cost of retaining superseded state and some review burden. Provenance integrity is not a certificate of scientific validity.

S10. REPRODUCIBILITY

Code availability. The ResearchLedger repository is at https://github.com/raja21068/ResearchLedger. Reproducibility procedures and snapshot identifiers for the reported validation are specified below.

A. Frozen identifiers

B. Commands

The synthetic mechanism suite and local validation harness require only Python and `pytest`; the confirmatory old/new paper-agent study will additionally require the selected repositories, their dependencies and the chosen paper-agent generator. No model API, GPU or network access is needed for the already executed local replays. From the `code/` directory:

```
PYTHONPATH=. pytest -q tests
PYTHONPATH=. python eval/research_state_mutation_benchmark.
    py --trials 1000 --seed 7
PYTHONPATH=. python eval/research_state_sequence_benchmark.
    py --episodes 100 --steps 25 --seed 17
PYTHONPATH=. python eval/
    research_state_concurrency_benchmark.py --trials 500 --
    seed 31
PYTHONPATH=. python eval/history_tamper_benchmark.py --
    trials 250 --seed 23
PYTHONPATH=. python eval/dependency_registration_benchmark.
    py --seed 41 --claims 20 --trials-per-point 200
PYTHONPATH=. python eval/support_set_diagnostic.py --sup-n
    300 --reg-n 100 --seed 61
PYTHONPATH=. python eval/state_machine_conformance.py --
    episodes 150 --steps 40 --illegal-trials 500 --seed 53
PYTHONPATH=. python eval/tms_crosscheck.py --n 300 --seed
    61
PYTHONPATH=. python eval/scaling_v13.py --scales 1000 10000
     50000 100000 --extra 10000:5 10000:20 50000:5
PYTHONPATH=. python -m eval.rrt.run --selftest 100 --seed 5
         # synthetic self-test, NOT RRT results
PYTHONPATH=. python -m eval.rrt.run --data rrt_data/
               # real trajectories, once collected
# ResearchLedger v2.2 validation source root:
python -m pytest -q                          # may be
    chunked on short command budgets
PYTHONPATH=. python validation_study/run_semantic_replay.py
PYTHONPATH=. python validation_study/analysis/
    expanded_semantic_replay.py
PYTHONPATH=. python validation_study/trajectory_runner.py
    --fresh
PYTHONPATH=. python validation_study/analysis/
    transaction_fault_injection.py
PYTHONPATH=. python validation_study/analysis/
    bootstrap_repository_clustered.py
# Blind human packets / aggregation are generated by
    validation_study/annotation_tool.py
PYTHONPATH=. python ../eval/
    researchledger_v2_validator_benchmark.py --trials-per-
    scenario 50
```

The RRT tooling is `eval/rrt/` (schema, independent oracle, systems, metrics, agreement, blind packets); protocols are `docs/RRT_PROTOCOL.md` and `docs/HUMAN_AUDIT_STUDY.md`; the audit-study analysis is `eval/analyze_audit_study.py`.

REFERENCES

[1] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[2] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychological Bulletin*, vol. 76, no. 5, pp. 378–382, 1971.

[3] J. Miao, J. R. Davis, Y. Zhang, et al., "Reimagining research papers as interactive and reliable ai agents," *Nature*, 2026. DOI: 10.1038/s41586-026-11044-y [Online]. Available: https://doi.org/10.1038/s41586-026-11044-y

[4] R. Meng, B. D. Mishra, J. Chen, et al., *Scientistone: Towards human-level autonomous research via chain-of-evidence*, Preprint; not peer reviewed, 2026. arXiv: 2605.26340 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2605.26340

[5] Z. Ren, R. Li, X. Zhang, et al., *Evigraph: Evidence-guided autonomous research agents*, Preprint; not peer reviewed, 2026. arXiv: 2608.04738 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2608.04738

[6] R. Yang, Y. Li, and S. Li, *Aris: Autonomous research via adversarial multi-agent collaboration*, Preprint; not peer reviewed, 2026. arXiv: 2605.03042 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2605.03042

[7] J. He and D. Yu, *Beyond memory: A transactional continuity kernel for long-lived ai agents*, Preprint; not peer reviewed, 2026. arXiv: 2608.11632 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2608.11632

[8] J. Doyle, "A truth maintenance system," *Artificial Intelligence*, vol. 12, no. 3, pp. 231–272, 1979. DOI: 10.1016/0004-3702(79)90008-0

[9] J. de Kleer, "An assumption-based tms," *Artificial Intelligence*, vol. 28, no. 2, pp. 127–162, 1986. DOI: 10.1016/0004-3702(86)90080-9

[10] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Database Theory — ICDT 2001*, ser. Lecture Notes in Computer Science, vol. 1973, 2001, pp. 316–330. DOI: 10.1007/3-540-44503-X_20

[11] T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance semirings," in *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2007, pp. 31–40. DOI: 10.1145/1265530.1265535